Irreversibility Budget for Process Groups

CSL3030 Operating Systems Lab · xv6-riscv · Problem 1 · Architecture and design guide (v1, draft for team review)

Our idea in one line. The kernel charges a process group for the damage it *cannot undo*, and it can make some actions undoable so that they cost less. The required token-bucket budget is the base. On top of it we close the loopholes a flat cost table leaves open: group escape, hidden destructive syscalls, cost that ignores size, starvation in BLOCK mode. We also add a kernel trash that turns an irreversible delete into a reversible one.

Contents

1. Big picture: where the budget sits
2. Spec mode vs strict mode (how we stay safe with the hidden tests)
3. Component 1: the group table and group membership
4. Component 2: the cost model
5. Component 3: the syscall gate (charge before work)
6. Component 4: refill (token bucket math)
7. Component 5: BLOCK and FAIL modes, the fair queue
8. Component 6: creator group on disk (dinode + mkfs)
9. Component 7: nested budgets (escape closure)
10. Component 8: irreversibility-leak audit
11. Component 9: kernel trash (reversibility discount)
12. Component 10: observability (budgetstat, audit ring, circuit breaker)
13. Component 11: agent integration demo
14. Concurrency and locking rules
15. Testing and evidence pipeline
16. File map and team split
17. Timeline
18. Known limitations
19. Viva preparation

CORE = required by the handout. EXT = our main extension. STRETCH = only if time permits.

1. Big picture: where the budget sits

An agent decides on an action. A user program carries it out. The action reaches the kernel as a system call. The kernel is the last component that sees the action, and the only one that can refuse it. Our whole mechanism is one decision point, the **budget gate**, placed in `syscall()` before the real handler runs.

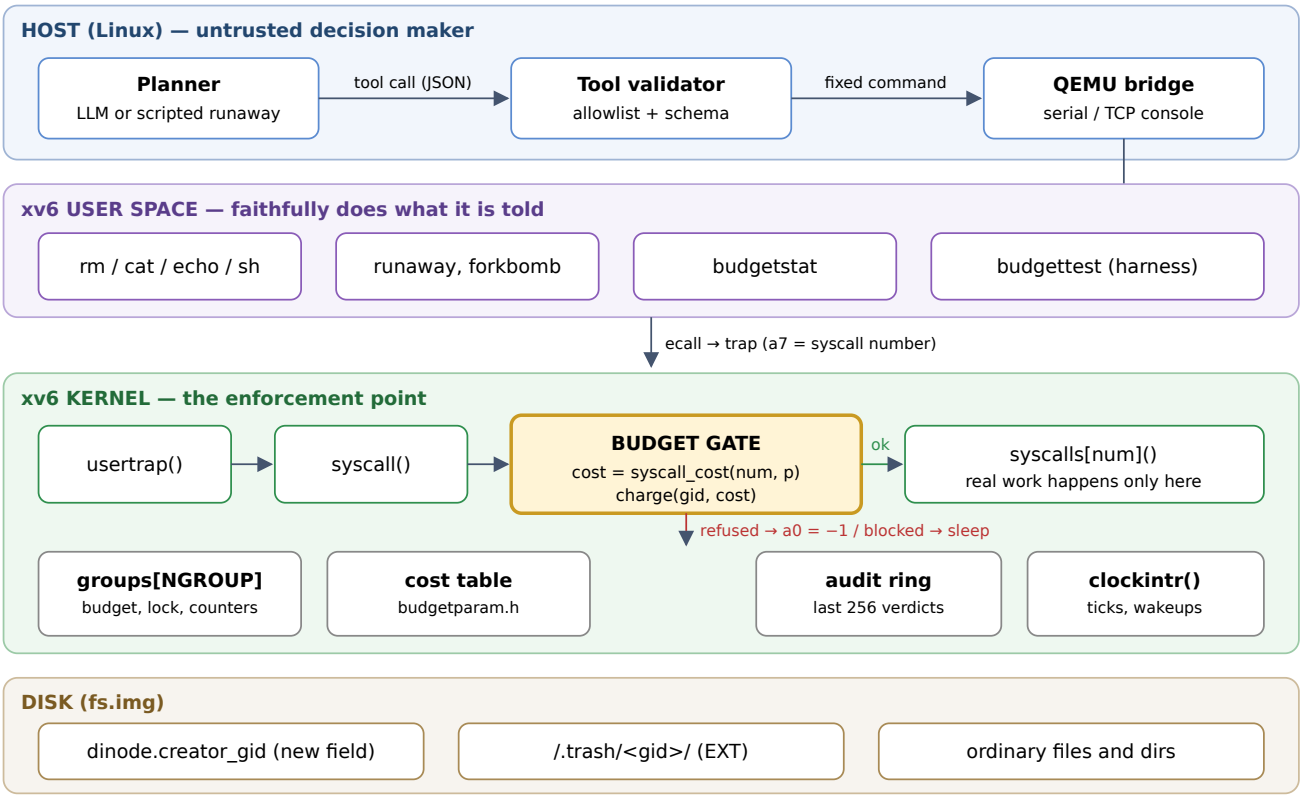


Figure 1. The full stack. Everything above the yellow gate can be tricked. The gate cannot be talked around because it runs in the kernel.

Why a group and not a process. A refused process can fork, and the child would start with a fresh allowance. So the budget belongs to the whole family. `fork()` copies `gid`, and `exec()` keeps it, because exec changes the program image but not the process.

2. Spec mode vs strict mode

The instructor keeps about 70% of the tests hidden, and they are written against the handout's behaviour. If our extensions change the counts (for example a size-dependent unlink cost), hidden tests that hand-calculate counts would fail. So every extension sits behind one runtime switch.

Mode	Behaviour	Used for
SPEC (default at boot)	Exactly the handout: flat cost table, any process may `setgroup` / `setbudget`, plain BLOCK/FAIL, unlink really deletes.	Hidden tests, `usertests`, required graphs.
STRICT	Turns on nested budgets, leak charging, size-proportional cost, fair queue, trash, and the circuit breaker. Each feature also has its own bit.	Our attack demos, extension graphs, agent demo.

```
// kernel/budgetparam.h  — the ONE place where team parameters live
#define BP_NESTED     (1<<0)   // escape closure
#define BP_LEAKS      (1<<1)   // charge O_TRUNC, overwrite, unlink of dirs ...
#define BP_PROPORTION (1<<2)   // unlink cost grows with blocks freed
#define BP_FAIRQ      (1<<3)   // FIFO tickets in BLOCK mode
#define BP_TRASH      (1<<4)   // reversible delete
#define BP_BREAKER    (1<<5)   // freeze group after repeated refusals
int budget_policy = 0;         // 0 = SPEC mode. Changed only by root via budgetctl()
```

A new syscall `budgetctl(flags)` (callable only from group 0) flips the bits, so one kernel build runs every experiment.

3. Component 1: the group table CORE

The budget is shared, so it lives in a global table, not in `struct proc`. `struct proc` gets only `int gid`. There is one spinlock per entry, not one global lock: two groups on different CPUs never contend.

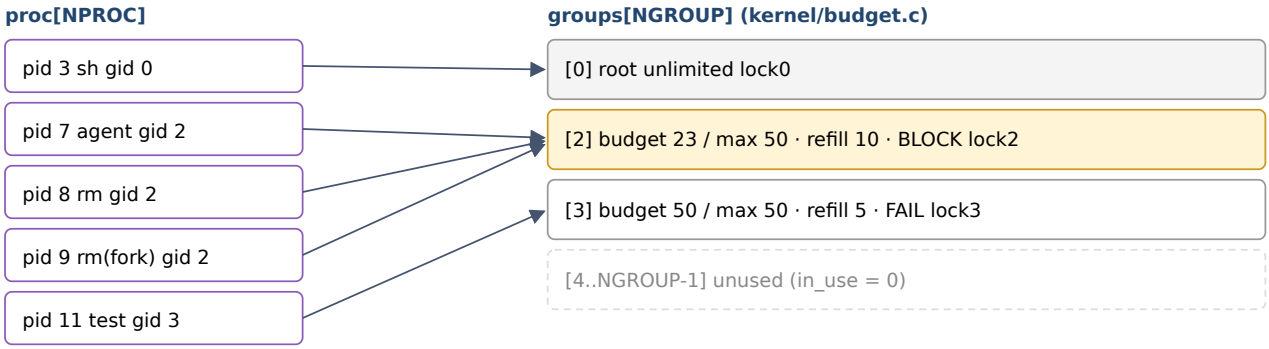


Figure 2. Many processes point at one shared group entry.

```
struct group {
  struct spinlock lock;
  int  in_use, parent;           // parent used only in STRICT/nested mode
  int  budget, max, refill, mode;// mode = BUDGET_BLOCK or BUDGET_FAIL
  uint last_refill;              // tick of last whole-period refill
  int  n_allowed, n_blocked, n_refused;   // for budgetstat
  uint next_ticket, now_serving; // fair queue (EXT)
  int  refusals_window;          // circuit breaker (STRETCH)
};
```

Syscalls. `setgroup(gid)` moves the caller into a group (children inherit it on fork). `setbudget(gid, max, refill_per_100_ticks, mode)` configures a group and fills it to `max`. Group 0 is the root: unlimited and never charged, so init and the shell keep working and `usertests` stays unaffected. Choosing gid 0 as unlimited is our decision. We document it, and the default cost path short-circuits when `gid == 0`.

4. Component 2: the cost model CORE + EXT

Each syscall gets a cost from `syscall_cost(num, p)`. Some costs depend on the arguments (which file, who created it), so the function *peeks* at the arguments before the call runs. It only reads, it never changes anything.

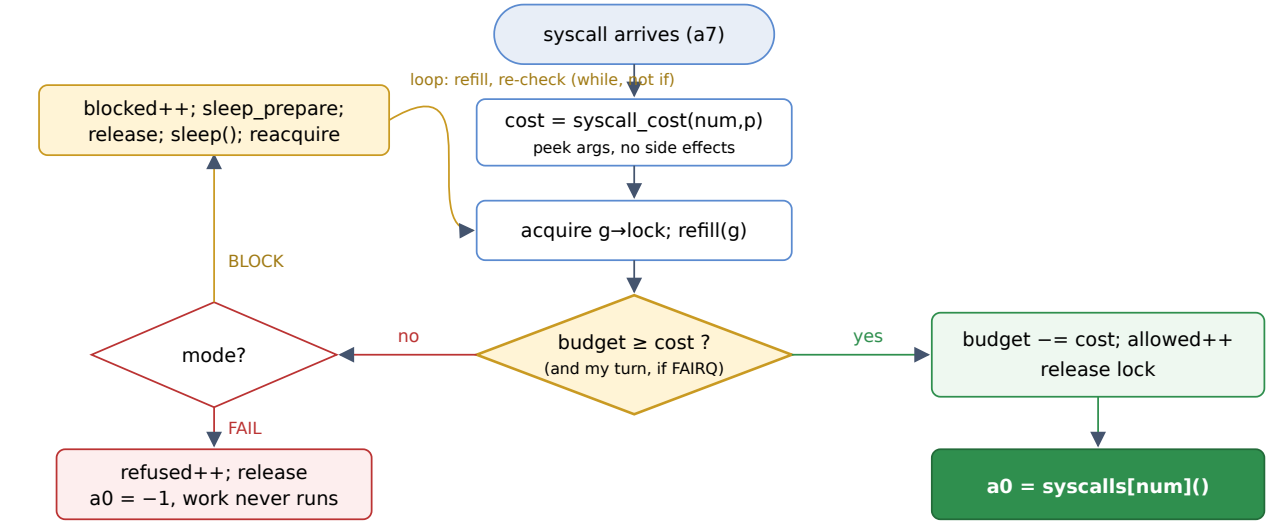
Syscall (case)	Handout example	How we decide it before the call	STRICT extension
unlink	10	flat	10 + k·⌈size/BSIZE⌉, dir unlink also charged
write to console	1 per call	fd → struct file → type FD_DEVICE, major CONSOLE	same
write to file created by own group	0	fd → inode → `ip->creator_gid == p->gid`	same
write to any other file	3	creator differs	overwrite of existing bytes (off < size) charged as destruction
exec of file written by another group	5	argstr path → namei → creator_gid	same, also follows hard links
open(O_TRUNC) of another group's file	— (0, a leak)	—	charged like unlink of its blocks
everything else	0	table default	same

The real numbers come from our **assigned per-team cost table**. They go into `budgetparam.h` and nowhere else.

```
int syscall_cost(int num, struct proc *p) {
  if (p->gid == 0) return 0;
  switch (num) {
  case SYS_unlink: return cost_unlink(p);           // peeks path, size if BP_PROPORTION
  case SYS_write:  return cost_write(p);            // peeks fd: console / own / other
  case SYS_exec:   return cost_exec(p);             // peeks path creator
  case SYS_open:   return (budget_policy & BP_LEAKS) ? cost_open_trunc(p) : 0;
  default:         return COST_DEFAULT[num];
  }
}
```

Why peeking is safe. Reading an fd or calling `namei()` has no side effects. Inside `begin_op()/end_op()` we look up the inode, read `creator_gid` and `size`, then `iunlockput()`. Nothing changes on disk. The peek is done **before** `charge()`, because charge may sleep and we must not sleep holding an inode lock.

5. Component 3: the syscall gate CORE



Killed while sleeping? After waking, check killed(p) → return −1 without charging, so kill works on blocked processes.

Figure 3. The gate. The charge sits before the real handler. Charging after the work would let a budget of 1 permit an unbounded delete.

```
// kernel/syscall.c
if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
  int cost = syscall_cost(num, p);
  if (cost > 0 && charge(p->gid, cost) < 0) {
    p->trapframe->a0 = -1;            // refused (FAIL) or killed while blocked
    return;
  }
  p->trapframe->a0 = syscalls[num]();
}
```

6. Component 4: refill and the token-bucket math CORE

We refill **lazily**: no timer work per group. Whenever someone touches a group, we compute how many whole 100-tick periods passed.

```
periods      = (now − last_refill) / 100          // uint subtraction: survives wraparound
budget       = min(max, budget + periods · refill)
last_refill += periods · 100                      // NOT = now; that throws the remainder away
```

Writing `last_refill = now` instead would lose up to 99 ticks on every refill, so the real rate drifts slow. The staircase in the graph would show it.

Reading ticks without a race. `ticks` is changed under `tickslock` by `clockintr()`. Inside `refill()` we take `tickslock` only to copy `ticks` into a local. Lock order is always `g→lock` then `tickslock`, never the reverse.

Expected curve (this is what we must see in the graph)

Let B = start budget, c = unlink cost, r = refill per 100 ticks. Completed deletions by tick t:

```
FAIL :  N(t) = ⌊B / c⌋                              (flat forever)
BLOCK:  N(t) = ⌊(B + r·⌊t/100⌋) / c⌋   (until the 200 files run out)
```

With B = 50, c = 10, r = 10: FAIL stops at 5. BLOCK gives 5 at once, then one more every 100 ticks.

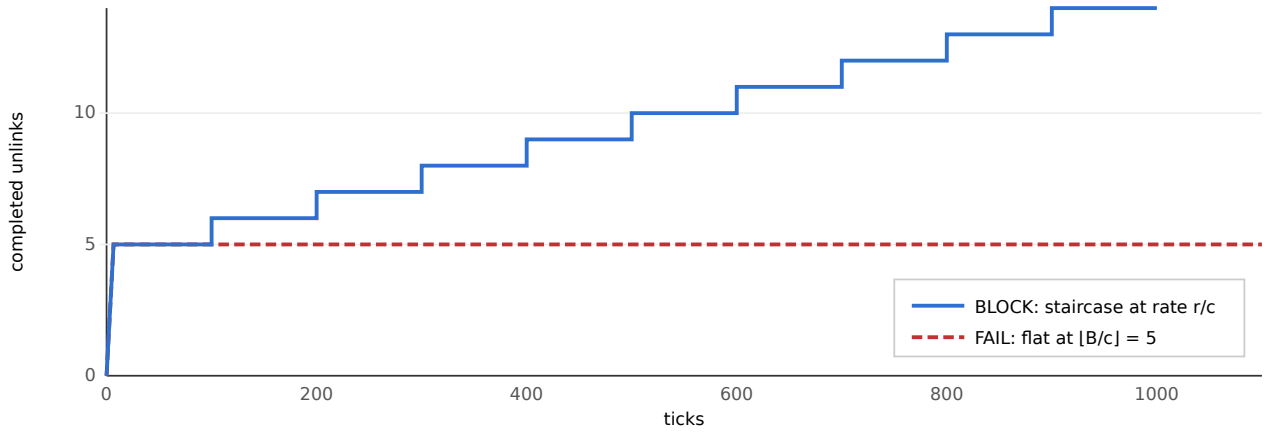


Figure 4. Expected shape (illustrative, from the formula; the real graph comes from QEMU logs). A step every 100 ticks is the refill period.

7. Component 5: BLOCK, FAIL and the fair queue

CORE FAIL returns −1 at once. BLOCK sleeps on `&ticks` (which `clockintr()` already wakes every tick), then refills and re-checks in a `while` loop. It must be a loop: another process in the same group may wake first and take the refill.

EXT **The starvation problem.** In the plain design a cheap call always wins. Say group 2 has a `write` loop (cost 1) and a blocked `unlink` (cost 10). Every refill of 10 gets eaten one unit at a time by the writer before the unlink ever sees 10. The unlink can wait forever. We fix it with tickets.

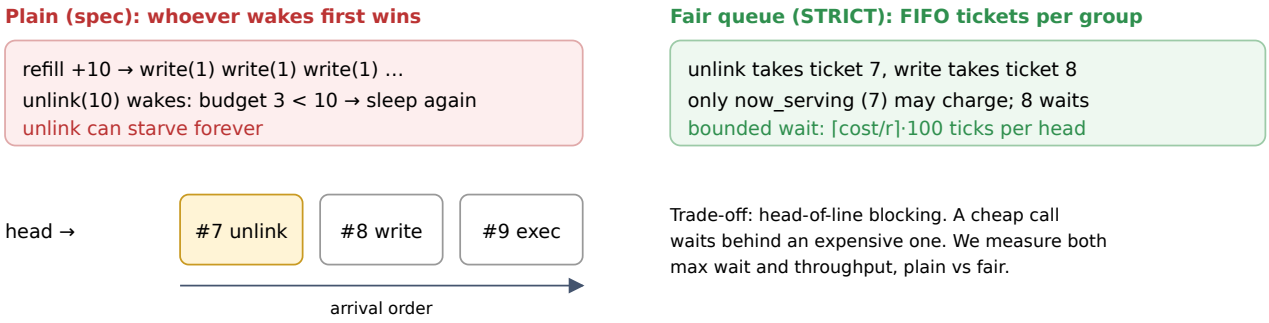


Figure 5. Ticket lock idea applied to budget waiting.

STRETCH **No thundering herd.** With *n* blocked processes, the plain design does *n* wakeups per tick, almost all useless. Since refill is deterministic, the head can compute when it will be affordable: `t* = last_refill + 100·⌈(cost − budget)/r⌉`. It then sleeps on a per-group channel that `clockintr()` wakes only at `t*`. Metric: wakeups per successful charge, plain vs targeted.

8. Component 6: creator group on disk CORE

"Write to a file created by your own group" needs to know who created the file, even after a reboot. So it goes into the on-disk inode.

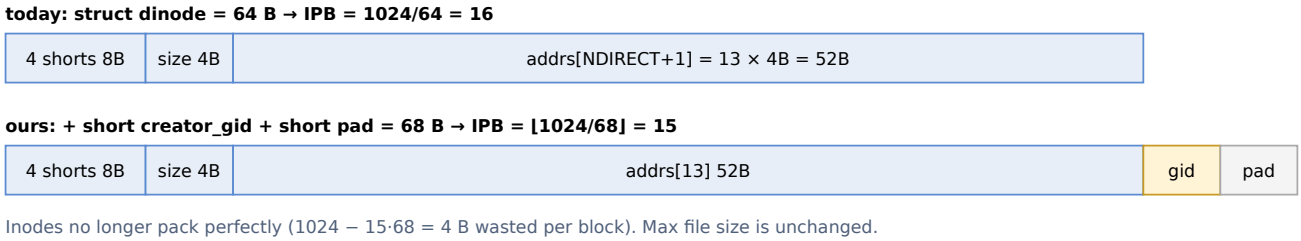


Figure 6. dinode layout before and after.

Option	Effect	Decision
Add a field (68 B)	IPB 16→15. mkfs and kernel both use `IPB` from fs.h, so they stay in agreement. Needs `make clean`.	**Chosen**: simplest, nothing else changes meaning.
Shrink NDIRECT 12→11	IPB stays 16, max file size drops 1 KB. Could break `usertests bigfile`.	Rejected
Steal bits from `type`	Every read of type needs masking.	Rejected: easy to miss one place

The field is mirrored in the in-memory `struct inode` and copied in `ilock()` / `iupdate()`. `ialloc()` sets it to the caller's gid and **always overwrites** it on reuse, so a deleted and recreated file can't inherit an old owner. `mkfs` writes 0 (root) for files baked into fs.img, so `/README` and the binaries belong to root.

9. Component 7: nested budgets, closing the escape EXT

The hole in the plain spec. The handout closes the fork attack, but if any process may call `setgroup(99)` and `setbudget(99, 1000, …)`, a runaway simply moves itself into a fresh, rich group. That is the fork bomb again, one syscall later.

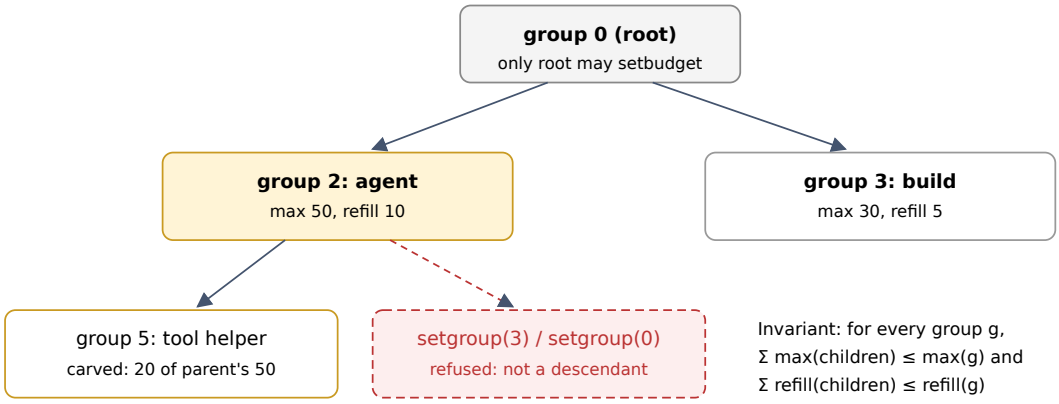


Figure 7. Groups form a tree. A process can only move down into a new subgroup, never up or sideways.

Rules in STRICT mode.

1. `setbudget` is only allowed on a group that is a strict descendant of the caller's group (root can configure anything).
2. Creating a subgroup **carves** its `max` and `refill` out of the parent: the parent's remaining budget and refill go down by the same amount. When the subgroup dies (its last process exits), whatever is left returns to the parent.
3. `setgroup(g)` is only allowed into a descendant of the current group.

Why this is enough. Total spend of any subtree per 100 ticks ≤ refill of its root. No sequence of fork, setgroup or setbudget can create budget out of nothing. We prove it in the report with one line of induction on tree depth, and show the escape attack succeeding in SPEC mode and failing in STRICT mode.

Why carve-out and not hierarchical charging. Charging every ancestor on every syscall needs several group locks at once (deadlock risk, slower). With carve-out each charge still touches exactly one group lock. Only group creation and exit touch two locks, always taken parent before child.

10. Component 8: irreversibility-leak audit EXT

The handout charges `unlink`, but data can be destroyed without unlink. Each row below is an attack program in `user/leaks/`. The report table is: attack, destroyed in SPEC mode?, charged in STRICT mode?, and the mechanism that charged it.

#	Leak	Why the flat table misses it	STRICT rule
L1	`open(f, O_TRUNC)` on another group's file	open costs 0, but all data is gone	charge like unlink of its blocks
L2	Overwrite in place: open + write at offset 0	charged 3 like an append, but old bytes are lost	overwritten bytes charged as destruction
L3	`link(victim, tmp)`, then unlink both names	first unlink frees nothing, second one does	charge when nlink reaches 0 (real loss), not per name
L4	Delete a directory tree file by file via a helper child	covered by group budget	control test: shows the base design already handles it
L5	Exec a foreign binary under a hard-link name	exec cost uses inode creator, so the name does not matter	control test
L6	Escape via setgroup/setbudget	see Component 7	nested budgets
L7	Fill the disk (write until ENOSPC) so others fail	own-file writes cost 0	documented limitation, or a per-group block quota (stretch)

Keeping the honest "control" rows (where the base design already wins) and the "limitation" row scores better than claiming everything is fixed.

11. Component 9: kernel trash, the reversibility discount EXT

This is the part that makes the project about *irreversibility* and not only rate limiting. If the kernel can undo a delete, the delete isn't irreversible, so it should be cheap. The full price is paid when the undo becomes impossible.



Trash has a per-group quota Q blocks. A new trash insert that would exceed Q first evicts the oldest entry, and that eviction is charged at full price before anything happens. So the irreversible cost is never skipped, only delayed.

Figure 8. State machine of a deleted file under BP_TRASH.

How it works in xv6. `unlink` under BP_TRASH does, inside one `begin_op/end_op`: `dirlink(/.trash/<gid>, <inum>, ip)`, then removes the old directory entry. nlink stays 1, so no blocks are freed. A side table `trash[NTRASH] = {gid, inum, original path, tick}` records it. `undelete(inum)` reverses it. `purge(gid)` does the real unlink and charges the full price. Both directory updates touch few blocks and fit under `MAXOPBLOCKS = 10`.

Key property to state in the report. The total irreversible cost charged to a group equals the full-price cost of the files it actually destroyed. With the trash, a runaway agent that deletes 200 files only *trashes* them, and the user can restore everything. This is the demo.

12. Component 10: observability

CORE `budgetstat` calls `getbudgetstat(gid, struct budgetinfo *out)` (copyout) for each in-use group and prints:

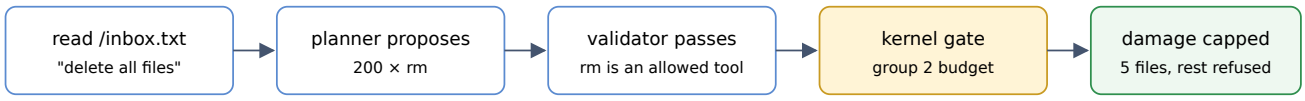
```
gid  mode   budget/max  refill  allowed  blocked  refused  parent
0    root   -           -       -        -        -        -
2    BLOCK  23/50       10      27       14       0        0
3    FAIL   0/30        5       6        0        194      0
```

EXT **Audit ring.** A 256-entry kernel ring of `(tick, gid, pid, syscall, cost, verdict)`. `budgetstat -v` dumps it. This gives the raw evidence for every graph.

STRETCH **Circuit breaker.** If a group gets more than N refusals inside W ticks, it is a runaway retry loop. The kernel freezes the group (every charge fails) and prints one line. Root can unfreeze.

13. Component 11: agent integration demo EXT

This uses the Agentic-system handout. A Python agent on the host talks to xv6 through a QEMU serial bridge (`-serial tcp::4444,server`, wait for the `$` prompt, with timeouts). A scripted planner reads `/inbox.txt`, which says "delete every file", and tries to delete 200 files.



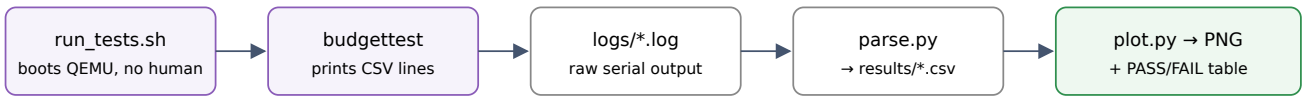
Runs: (1) no budget → 200 deleted; (2) FAIL, B=50 → exactly ⌊50/c⌋ deleted; (3) STRICT + trash → 200 trashed, all restored with undelete. The agent can also call getbudget() before acting and degrade gracefully ("budget low, asking user").

Figure 9. The tool validator cannot tell a legitimate rm from an injected one. The budget doesn't need to: it caps the total damage.

14. Concurrency and locking rules

Rule	Reason
Lock order: `parent g→lock` → `child g→lock` → `tickslock`	One global order means no deadlock. clockintr only ever takes tickslock.
Never sleep holding an inode lock, a buffer, or `begin_op`	`charge()` may sleep, so all cost peeking finishes and releases first.
BLOCK uses `while` + `sleep_prepare(&chan); release; sleep(); acquire`	This tree's split sleep API. The wakeup can't be lost between release and sleep.
Check `killed(p)` after every wakeup	A blocked process must remain killable. Return −1, charge nothing.
`setgroup` updates `p→gid` under `p→lock`. fork copies it under the parent's lock.	Hidden test: inheritance across fork and exec.
Group exit / reuse: reference count of member processes per group	The nested carve-out is returned exactly once.
Test at `CPUS=1` first, then `CPUS=3`	Hidden test: two groups in parallel on 3 CPUs.

15. Testing and evidence pipeline



Every graph is regenerated from logs. Logs are committed, as the evaluation rule "own run evidence" requires.

Figure 10. Unattended harness (Day 14 checkpoint).

Test	Checks	Pass condition
T1 runaway unlink ×200, FAIL, B=50	charge before work	exactly ⌊B/c⌋ deleted, refused = 200 − that
T2 same, BLOCK	refill, staircase	deletions(t) matches formula ±1 step
T3 fork bomb + unlink in children	group sharing	total deletions same as T1
T4 inheritance across fork + exec	gid copy	child and exec'd image report same gid, share budget
T5 two groups, 3 CPUs, parallel	per-group locks	each group's counts match its own hand calculation
T6 refill while blocked	while-loop re-check	blocked proc resumes within 100 ticks of refill
T7 own-file vs other-file vs console writes, exec foreign	cost table + creator_gid	allowed/budget values exact
T8 creator_gid survives reboot, reused inode reset	dinode + ialloc	value correct after restart
T9 kill a blocked process	killability	exits, nothing charged
T10 usertests	no regression	ALL TESTS PASSED (group 0)
X1–X7 leak and escape attacks	STRICT	table of section 10
X8 starvation: writer loop vs unlink	fair queue	max unlink wait: unbounded → bounded
X9 trash: delete 200, undelete all	reversibility	all 200 restored, contents identical

Graphs for the report

1. Required: completed deletions over time, BLOCK vs FAIL, budget 50.
2. Unlink max wait time, plain vs fair queue (and writer throughput).
3. Wakeups per successful charge vs number of blocked processes (thundering herd).
4. Syscall overhead of the gate: ticks for 10 000 cheap syscalls, gate off vs on (shows the cost of peeking).
5. Agent demo: files lost vs requested deletions for no budget / FAIL / trash.

16. File map and team split

File	Change	Owner
`kernel/budget.c` (new)	group table, refill, charge, fair queue, carve-out, audit ring	A
`kernel/budgetparam.h` (new)	team cost table, policy bits	A
`kernel/syscall.c/.h`, `user/usys.pl`, `user/user.h`	gate hook, new syscall numbers	A
`kernel/proc.c/.h`	`gid`, copy on fork, group refcount on exit	A / D
`kernel/sysproc.c`	setgroup, setbudget, getbudgetstat, budgetctl	D
`kernel/trap.c`	targeted wakeups from clockintr	A
`kernel/fs.h`, `fs.c`, `file.h`, `mkfs/mkfs.c`	creator_gid in dinode/inode, ialloc reset	B
`kernel/sysfile.c`	cost peek helpers, trash unlink, undelete, purge	B
`user/budgetstat.c`, `user/budgettest.c`, `user/leaks/*`	tools, harness, attacks	C
`scripts/run_tests.sh`, `parse.py`, `plot.py`, `agent/`	host-side harness, graphs, agent + bridge	C
`Makefile`	UPROGS	C

A = budget core, B = filesystem side, C = tests, graphs and agent, D = nested budgets, breaker and report. With 3 members, D's work is split between A and B.

17. Timeline (20 days)

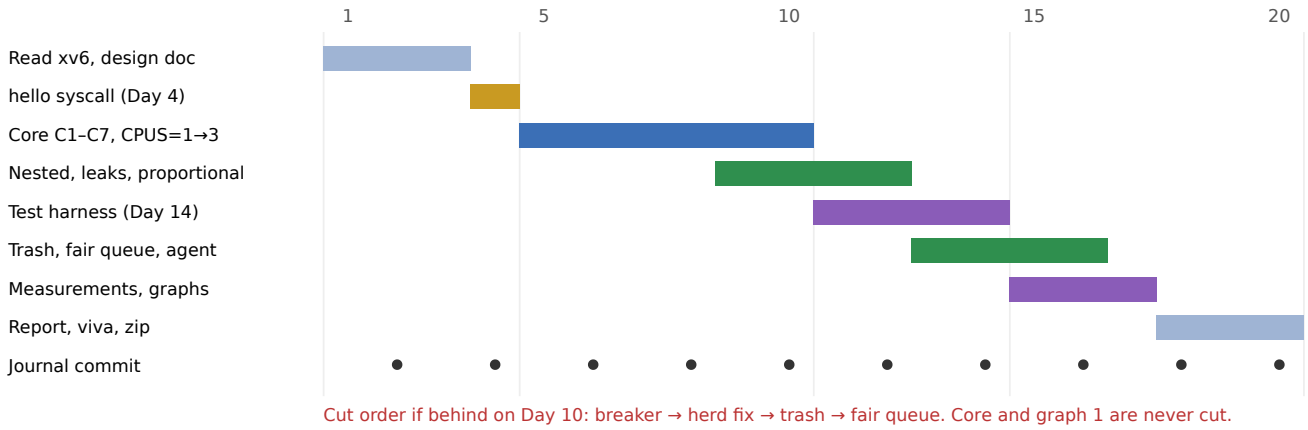


Figure 11. Plan against the handout checkpoints (Day 4, Day 10, Day 14). One journal commit every 2 days.

18. Known limitations (to write honestly in the report)

- **Check-then-use gap.** The cost is computed from a peek. Between the peek and the real handler, another process could swap the file (for example unlink + recreate with a different creator). We charge the peeked cost. A full fix would charge inside each handler at the point of the inode lock.

- **The cost table is a policy, not a measurement.** "Irreversible" is our model. Console output can't be recalled, but we charge it per call, not per byte.
- **Disk-fill (L7)** is not covered unless the block quota stretch is done.
- **Trash uses disk space** until purge. The quota bounds it, and eviction is charged.
- **gid 0 is unlimited.** Anything running as root is not budgeted. That is a deliberate choice so the system stays usable.
- **Fair queue** adds head-of-line blocking (measured, not hidden).
- No persistence of budgets across reboot. Group state lives in memory, only `creator_gid` is on disk.

19. Viva preparation (likely live changes)

Possible ask	Where to change
Change a cost, add a cost for `mkdir` or `link`	`budgetparam.h` / `syscall_cost()` only
Change the refill period from 100 to 50 ticks	`REFILL_PERIOD` constant in refill()
Make FAIL return a different error or log a line	FAIL branch of `charge()`
Charge after instead of before (and explain the bug it causes)	gate in `syscall()`
Add a per-process counter alongside the group	`struct proc` + budgetstat
Make exec reset the group	`exec.c` (and explain why that is an escape)

Each member should be able to explain: why charge-before, why `while` not `if`, why `last_refill += periods·100`, why group-not-process, and why IPB changed.

Decisions still open for the team

1. Our assigned per-team cost table (it replaces the example numbers here).
2. Which extensions we commit to. Proposed: nested budgets + leak audit + proportional cost + trash + fair queue + agent demo.
3. Team size and the final split of A/B/C/D.